

A \$300,000 Mirage: Quantifying and Eliminating Look-Ahead Bias in an Intraday Futures Backtest

Drew Mann — github.com/manndrew9441-max/FuturesBot

July 2026

Abstract

Backtesting is only trustworthy if the simulated strategy receives data in the same order that it would arrive live. This paper documents a case study in which a single violation of that rule — the time at which the strategy sees a five-minute price bar — makes an essentially break-even intraday futures system look like one that earned over \$300,000 across five years. Using five years of one-minute Micro E-mini S&P 500 (MES) futures data (2021-2026; 1.77 million front-month bars), I run the same exact strategy code under two bar-delivery regimes that differ only in event ordering. The realistic, “live” regime yields a net P&L of -\$159.51 (profit factor 1.00); the look-ahead regime yields +\$309,783.83 (profit factor of 2.43) and lifts the win rate from 35.3% to 55.6%. I describe the mechanism, present a system architecture in which the same engine drives live trading, replay, and backtesting to guarantee parity by construction, and argue that look-ahead bias of this kind is both easy to introduce and difficult to spot.

Introduction

A backtest estimates how a trading strategy *would have* performed on historical data. Its value rests solely on one assumption: at every decision point, the backtesting engine is provided with only the exact information that would have been available during live trading at that moment. When a backtest uses any amount of future information to make a present decision, the result is **look-ahead bias**, which completely falsifies the estimated performance.

Look-ahead bias is a silent killer. Nowhere does it announce itself; it hides in plain sight in the form of consistent results. The equity curve still looks like an equity curve; the trade log still looks like a trade log; the P&L, win rate, profit factor, drawdown — all internally consistent. No single piece is obviously broken. The strategy simply appears better than it really is, sometimes dramatically so, other times not so much.

This paper presents a fully quantified, textbook instance discovered while developing an automated intraday futures strategy. The bias appeared because of a simple and easily overlooked detail: the order in which one-minute and five-minute price bars are handed to the strategy’s engine. I show that correcting this single order flow decision while changing nothing else about the entire system is the difference between a strategy that appears to have an industry-leading edge and one that is no better than flipping a coin.

Contributions

1. I isolate and quantify a look-ahead bias driven purely by bar-delivery timing, and measure its effect across five years of real futures data (Section 5).

2. I describe precisely why a naive delivery of multi-timeframe price bars allows for the leakage of future information, and give the exact code that does and does not leak (Section 3).
3. I present a parity-by-construction design in which live and backtest execution share one central engine, so that the backtest cannot diverge from live behavior without the divergence being explicit (Section 2).

The Trading System

The trading strategy that this system automates is an intraday, liquidity-based system for MES futures. Its internal logic is a state machine that looks for a specific sequence of market events to unfold: a sweep of a prior liquidity level, followed by a break of market structure, a retracement, and finally an entry.

WAITING $\rightarrow$ SWEPT $\rightarrow$ CONFIRMED_SHIFT $\rightarrow$ RETRACED $\rightarrow$ ENTRY

Upon entry the system submits a **bracket order**: a limit entry paired with a stop-loss and take-profit as a one-cancels-other (OCO) group. Position sizing is based on an internal risk calculator, and up to one trade is placed daily during a specific time window. The specifics of the trading logic are not the subject of this paper; what matters is that the engine is a fixed, deterministic function of the bars it is fed, so any change in results must be attributed to the bars and their ordering alone.

One engine, three modes. The central design decision is that a single strategy engine (`engine.py`) is the *only* instance of the trading logic. The system is composed of three different modes, each of which uses the same exact strategy.

- **Live:** a WebSocket feed delivers closed bars directly from the broker in real time.
- **Replay:** Previously captured live bars are re-fed in the order that they arrived in, reproducing a live session exactly.
- **Backtest:** historical bars downloaded through a market data source (in this case Databento) are streamed through the same engine.

Because there is no separate “backtest strategy” and “live strategy” the two cannot silently drift apart. This is what allows the bias to be measurable: the only thing that can vary is the bars themselves, and *how they are ordered before reaching the shared engine*.

Anatomy of the Bias

The strategy processes two different timeframes at once: one-minute bars and five-minute bars. A bar is only knowable (guaranteed not to change) once it has closed. In live trading:

- a one-minute bar opening at time T closes and is delivered at $T + 60$ seconds;
- a five-minute bar opening at time T closes and is delivered at $T + 300$ seconds.

So the five-minute bar spanning 09:00-09:05 does not exist, as a complete object, until 09:05. In particular, the one-minute bars at 09:00, 09:01, ..., 09:04 all arrive *before* the five-minute bar that contains them.

A naive backtest ignores this. It has every bar sitting in a table with an *open* timestamp, and here’s the problem: sorting events by open timestamp places the five-minute bar for 09:00 *before*

the one-minute bars that formed it. So, the engine is handed a completed five-minute bar at 09:00, whereas an engine running in real-time would not have seen said bar until 09:05. It sees the future.

The fix is simple: order events by *delivery time* (bar close), exactly as a live feed would. The listing below shows both orderings; they differ by one sort key.

```
events = [("5m", b) for b in day_5m] + [("1m", b) for b in day_1m]

if args.lookahead_mode:
    # BIASED: sort by OPEN timestamp -> a 5m bar is processed before
    # the 1m bars that formed it. The engine sees the future.
    events.sort(key=lambda x: x[1]["timestamp"])
else:
    # REALISTIC: sort by DELIVERY time = open + duration (close),
    # mirroring what the live websocket actually delivers.
    def _delivery_key(item):
        tf, b = item
        duration = 300 if tf == "5m" else 60
        return (b["timestamp"] + duration,      # when it would arrive live
                0 if tf == "5m" else 1,         # canonical tie-break
                b["timestamp"])
    events.sort(key=_delivery_key)
```

The realistic ordering is not merely good-looking; it matches the live delivery exactly. The production feed uses a debounced drainer that, at each delivery instant, orders simultaneously-arriving bars with the same canonical rule (five-minute before one-minute at a tie). With realistic mode, the backtester reproduces bar-for-bar what the live mode would have done. My very definition of “realistic” is the ordering that makes backtest identical to live and replay.

Experimental Design

Data. I use one-minute OHLCV bars for MES futures from a professional market-data vendor (Databento schema), covering 2021-04-23 through 2026-04-22. The raw file contains 2,660,042 rows across all listed contracts. I filter to a continuous front-month series — stitching the successive quarterly contracts and removing calendar-spread rows — yielding 1,769,291 one-minute bars. All 353,970 five-minute bars are synthesized from the one-minute data to ensure that both timeframes cannot disagree.

The controlled variable. The two runs are byte-for-byte identical in strategy code, data, date range, risk settings, and transaction costs. They differ in one and only one aspect: the event-ordering key shown above. This is a controlled experiment with a single independent variable.

Costs. Both runs charge realistic transaction costs: per-contract brokerage commissions and one tick of slippage per side. Costs are held identical across regimes, so they cannot explain the difference.

Account model. To measure the strategy’s raw statistical edge across the full period, both runs use an uncapped, continuously-trading account. (A separate “prop-firm” mode that halts on a maximum-loss limit exists for realism when simulating a funded evaluation, but it terminates trading on the first blow-up and is therefore inappropriate for a five-year edge measurement.)

Results

Table 1 presents the head-to-head comparison. Figure 1 shows the two equity curves.

Table 1: Identical strategy, identical data and costs, over five years of MES. The only difference is bar-delivery ordering.

Metric	Realistic (live-parity)	Look-ahead (biased)
Trades	643	711
Win rate	35.3%	55.6%
Net P&L	−\$159.51	+\$309,783.83
Profit factor	1.00	2.43
Average win	+\$1,419.87	+\$1,331.89
Average loss	−\$775.17	−\$684.54
Max drawdown	\$30,050.87	\$7,925.11
Return / drawdown	−0.01	39.09

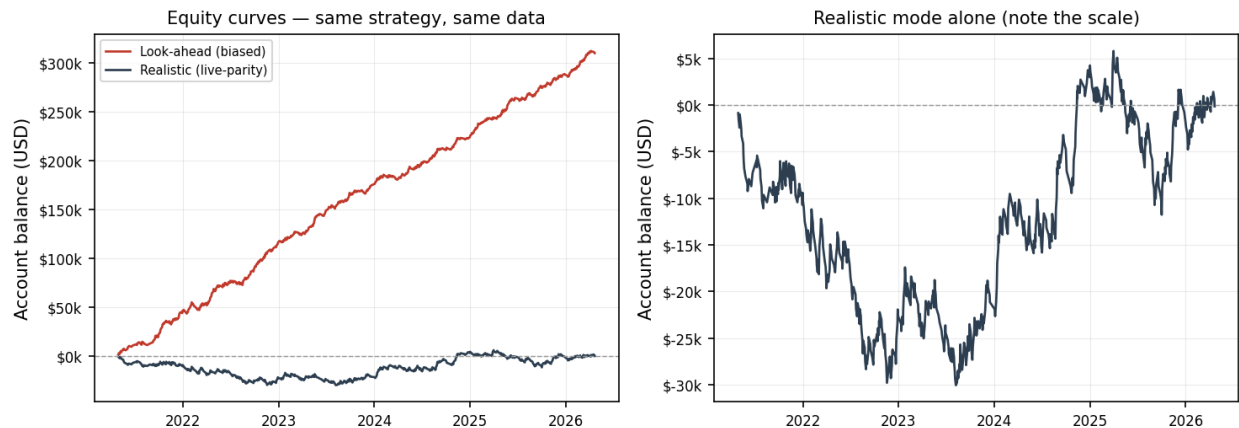


Figure 1: **Left:** the two equity curves on a shared axis. The biased run (red) compounds steadily to \$309,784; the realistic run (dark) never departs far from zero. **Right:** the realistic run alone, rescaled — its true path is a \$30,000 drawdown followed by a recovery back to break-even. The divergence is produced entirely by the sort key above.

The look-ahead regime does not merely inflate the total; it corrupts every statistic in an internally-consistent way. The win rate rises from 35.3% to 55.6% because the foreknowledge of the five-minute bar lets the engine avoid entries that were about to fail. Maximum drawdown *falls* from \$30,051 to \$7,925 — seeing the future is, unsurprisingly, good for drawdown control. Trusting these numbers leads to the belief that the strategy is remarkably smooth and profitable. This belief, however, is completely false.

Year by year. Table 2 breaks the realistic run down annually. The strategy’s break-even result is not solely due to profits and losses cancelling over a specific window; it fluctuates around zero year to year, which is what a genuine near-zero edge looks like.

Table 2: Realistic-mode results by year. The full-period total is -\$159.51

Year	Trades	Net P&L
2021 (from Apr)	80	-\$10,212.61
2022	139	-\$12,485.55
2023	124	+\$1,546.60
2024	125	+\$25,384.18
2025	136	-\$5,851.87
2026 (to Apr)	39	+\$1,459.74
Total	643	-\$159.51

Discussion

Why this is easy to get wrong. The biased ordering is the *natural one*. Given a table of bars with open timestamps, sorting by that timestamp is the first thing most people write, and it produces a plausible, monotonic event stream. The bug is not a crash or an exception; it is a default that happens to be wrong. Nothing in the output flags it, and in this specific case the per-trade statistics didn’t appear implausible. The only way to catch it is to ask, for every bar, “at what instant would this have been knowable live?” — and to enforce that answer in the event ordering.

Foreknowledge also changes behavior, not just outcomes. The biased run makes *more* trades (711 vs. 643), not just better ones. Because the engine’s state machine is fed a different event stream, it triggers on a different set of setups entirely. Look-ahead bias is therefore not a fixed “bonus” one could subtract off; it changes which trades exist. This is why it cannot be corrected after the fact and must be prevented in the data layer.

Parity as a safeguard. The reason this bias was found rather than shipped is the central-engine design of Section 2. Because replay re-feeds captured live bars through the same engine and must reproduce live results bar-for-bar, any ordering rule that disagrees with the live feed shows up as a replay mismatch. The realistic backtest ordering is simply the ordering that passes the parity check. Architecturally, making live and backtest share one code path converts a class of silent bias into a visible test failure.

Limitations

This is a single case study, not a general estimator of look-ahead bias, and several caveats apply.

1. The result covers one strategy on one instrument over one five-year window; the *magnitude* of the bias is specific to this setup, though the *mechanism* is general to any multi-timeframe, event-driven backtest.
2. I report descriptive performance statistics but do not attach confidence intervals or a formal significance test to the near-zero realistic edge; establishing that the strategy is statistically indistinguishable from break-even would require a bootstrap or block-resampling analysis.
3. The cost model (fixed commission plus one-tick slippage) is a reasonable approximation but does not model queue position, partial fills, or liquidity-dependent slippage.
4. Synthesizing five-minute bars from one-minute data is exact for OHLCV but assumes the one-minute source itself is clean.

5. The backtest’s historical data (Databento) is not identical to the data live and replay modes see (TopstepX WebSocket). A limited comparison of overlapping bars recoverable from the live session logs (18 closes) found an 83% exact agreement with all differences being exactly one tick in magnitude, consistent with known differences in how vendors measure bars. Some published sources suggest this can affect the backtest’s ability to perfectly model live trading [Concretum Group, 2026], but not the comparative result as both regimes use identical data.

None of these caveats affect the paper’s claim, which is comparative: the realistic and biased runs share all of them, so the gap between the two columns of Table 1 is attributable to ordering alone.

Conclusion

A profitable-looking backtest means nothing if it cheats, and the cheating can be as small as sorting events by the wrong timestamp. On five years of MES futures, a single bar-delivery ordering decision separates a strategy that appears to earn \$309,784 from one that actually earns −\$159. The defense is not vigilance alone but architecture: when live trading, replay, and backtesting are driven by one shared engine, and the backtest’s event ordering is calibrated to the live feed and verified by replay, look-ahead bias of this form becomes a test failure rather than a false fortune. The most valuable output of this project was not a trading edge — there isn’t one — but the apparatus that proved there isn’t.

Reproducibility

Both results are reproduced from the same script and data with a single flag difference:

```
# Realistic (live-parity) -- the honest result
python3 backtest_replay.py --file-1m "YOUR DATA FILE".csv \
    --start 2021-04-23 --end 2026-04-22 --fees

# Look-ahead (biased) -- identical, plus one flag
python3 backtest_replay.py --file-1m "YOUR DATA FILE".csv \
    --start 2021-04-23 --end 2026-04-22 --fees --lookahead-mode
```

Data availability. The MES one-minute OHLCV data used here is licensed from Databento (GLBX.MDP3, ohlcv-1m schema, symbol MES.FUT, 2021-04-23 to 2026-04-22) and is not redistributed with this repository, as CME market data licensing prohibits this. New Databento accounts receive free credits sufficient to obtain this data. The backtester accepts any CSV/Parquet source with standard OHLCV columns, so data from another vendor could in principle be used; this has not been tested.

References

Concretum Group (2026). *Backtesting Data Quality: Can Your Data Provider Be Trusted?* <https://concretumgroup.com/backtesting-data-quality-can-your-data-provider-be-trusted/>